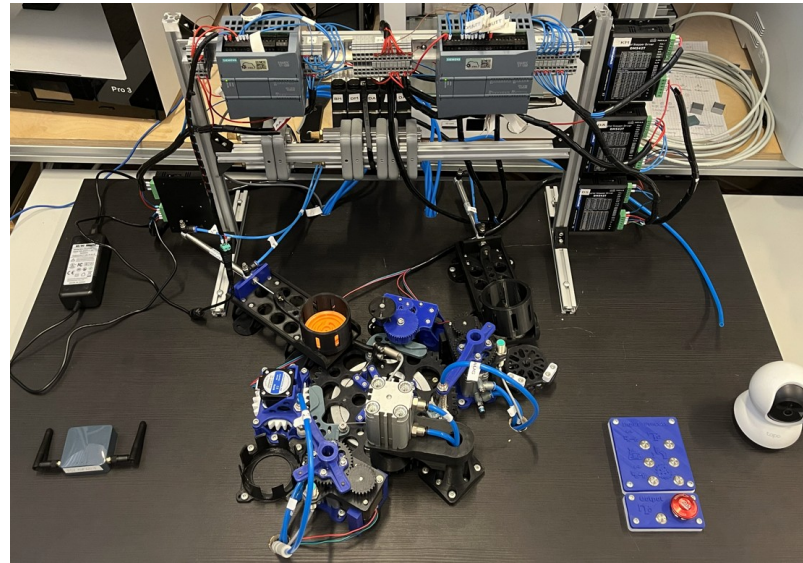


**HOCHSCHULE
HANNOVER**
UNIVERSITY OF
APPLIED SCIENCES
AND ARTS

–
Fakultät II
Maschinenbau und
Bioverfahrenstechnik



Anleitung zur Erstellung eines Digitalen Zwillings

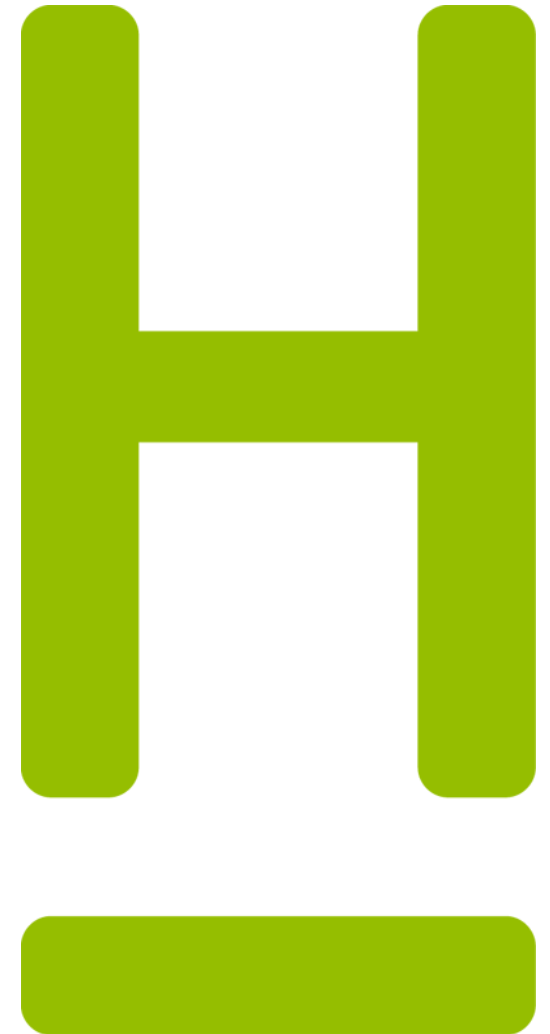
Projektarbeit von Tobias Küster 1690767

Betreut von:

Prof. Diersen

Dr. Yübo Wang

Herrn Ernst



Inhaltsverzeichnis

Kapitel 1	Einleitung	
Kapitel 2	Anwendungsfälle und Strukturdiagramme	<i>Folie 4</i>
Kapitel 3	Installation von Isaac Sim	<i>Folie 8</i>
Kapitel 4	Erste Schritte in Isaac Sim	<i>Folie 11</i>
	Oberfläche von Isaac Sim	
	Bewegliche-/ starre Teile	
Kapitel 5	Gelenke und Antriebe	<i>Folie 17</i>
	Dreh- und Schubgelenke mit Antrieb	
	Verschachtelte Gelenke	
Kapitel 6	Zu der Extension	<i>Folie 22</i>
	Ausführung von Quellcode in Isaac Sim	
	Einbinden der Extension	
	Lokaler OPC-UA Server	
Kapitel 8	Zu TIA Portal	<i>Folie 45</i>



Einleitung

Diese Anleitung beschreibt den methodischen Aufbau eines physikalisch korrekten Digitalen Zwillings in **NVIDIA Isaac Sim** am Beispiel der Anlage **Maze Runner**. Der Fokus liegt auf der Überführung von CAD-Baugruppen in eine funktionale Simulationsumgebung, wobei die Modellierung von Massenverteilungen, Außenhüllen sowie die Implementierung von Schub- und Drehgelenken und deren Antriebe im Vordergrund stehen. Zur Aktuierung der Gelenke wird eine bereitgestellte Extension genutzt, die über ein integriertes Control Panel Gelenke in Zielpositionen fährt. Während die reale Anlage über Siemens-SPS-Steuerungen und die Digital Twin App der Hochschule Rhein-Main betrieben wird, kann das hier erstellte Modell auch als alleinstehende Instanz zur fotorealistischen Visualisierung und physikalischen Validierung genutzt werden. Die Anleitung ist als lizenzfreier Step-by-Step-Guide konzipiert, erfordert keine Programmierkenntnisse und adressiert potenzielle Versionskonflikte der noch jungen Software durch spezifische Troubleshooting-Abschnitte am Ende jedes Kapitels.



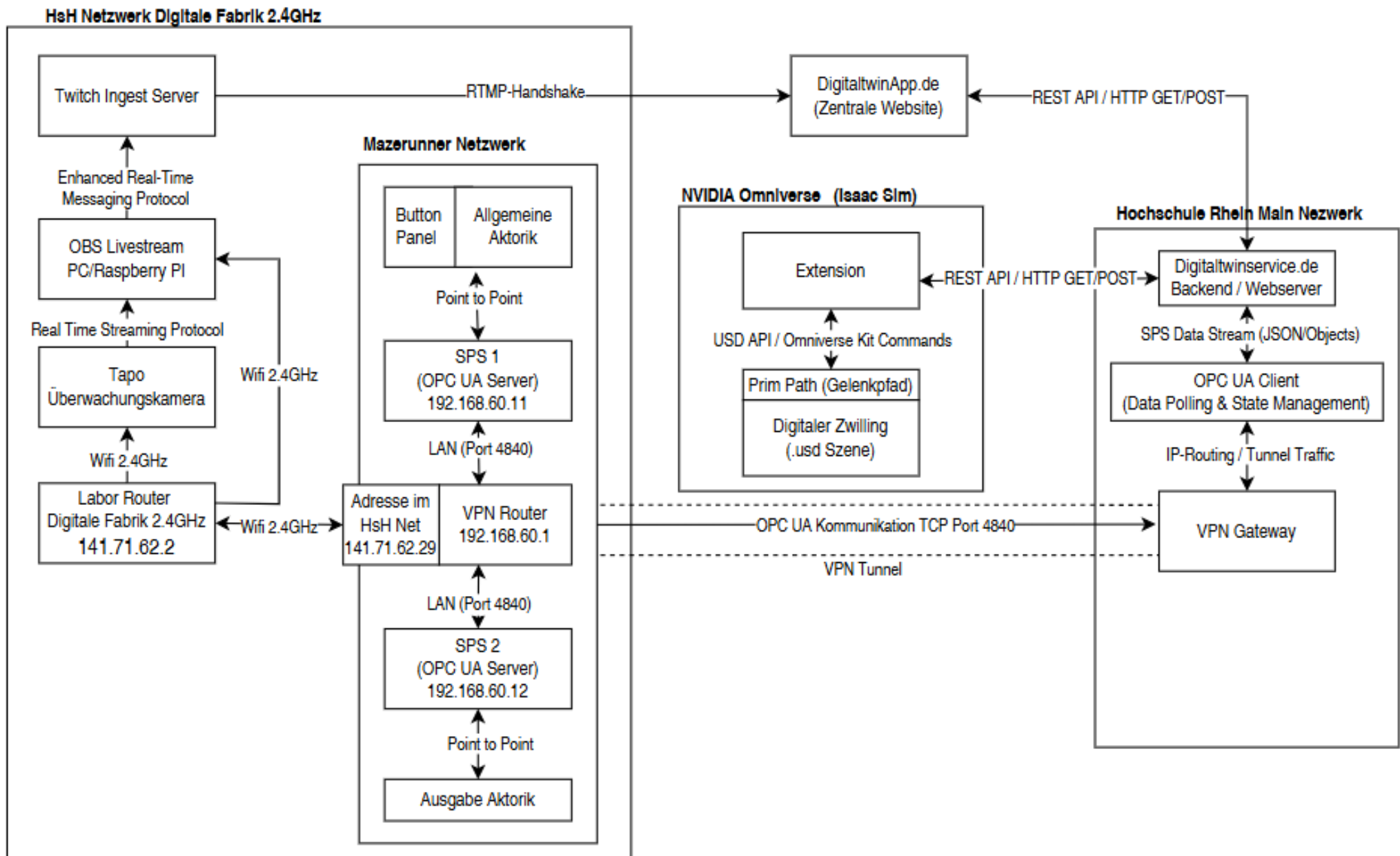
Use Case Matrix Variable setzen

Kriterium	Physikalisch (Labor)	Lokal an Anlage (OPC UA + Omni Extens.)	Omni Extens. via Digital Twin Service
User-Aktivität	Taster an der Anlage betätigen	Schaltfläche im Omniverse HMI betätigen	Schaltfläche im Omniverse HMI betätigen
Was passiert?	SPS-Variable direkt intern gesetzt	OPC UA Write Service	REST-API Call (HTTP POST) DigitalTwinService
Wo passiert es?	Innerhalb der SPS	Omniverse + lokales Netz	Omniverse + Internet/VPN
Wie passiert es?	Elektrisches Signal	Omniverse + lokaler OPC UA Server der SPS	HTTPS zu digitaltwinservice.de dann OPCUA via VPN
Aktorbewegung	Reale Hardware	Reale Hardware + Omniverse Szene	Reale Hardware + Omniverse Szene

Use Case Matrix Variable setzen

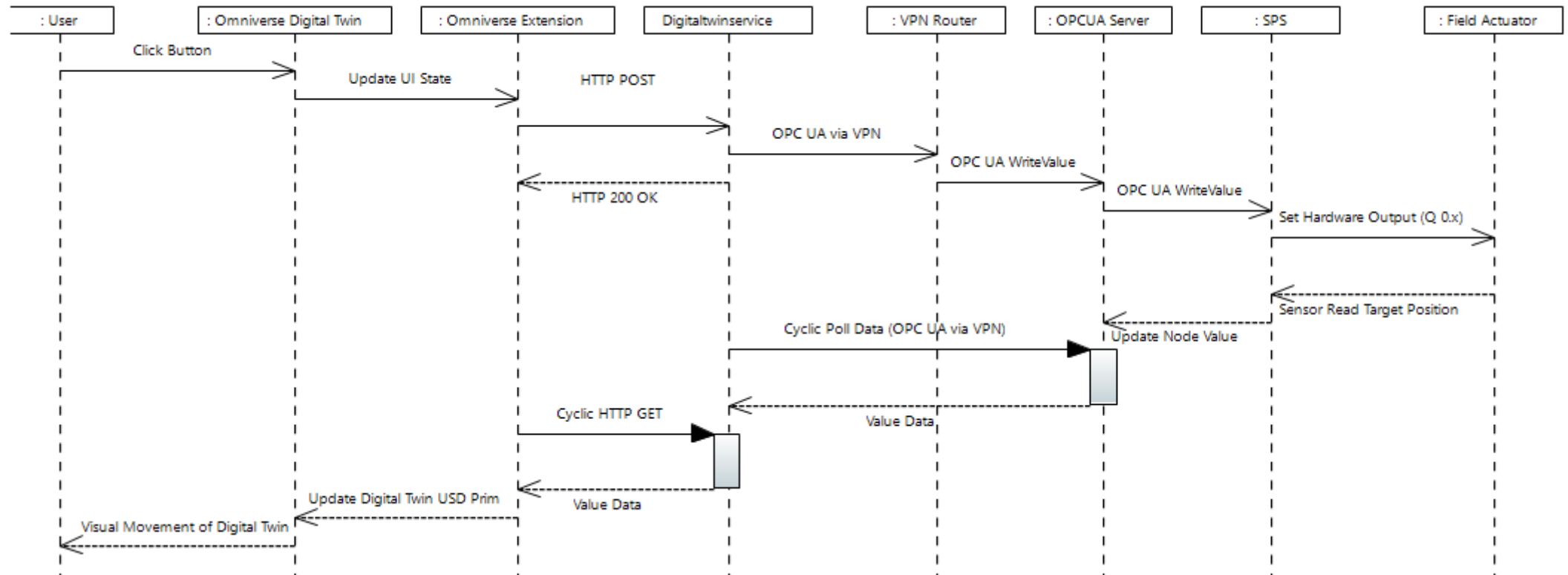
Kriterium	Physikalisch (Labor)	Lokal an Anlage (OPC UA + Omni Extens.)	Omni Extens. via Digital Twin Service
Rückmeldung	Status lokal an der Aktostellung der Anlage sichtbar	Omniverse Szene + ggf. Anlage	Omniverse Szene + ggf. Kamera Lifestream
Kommunikation	Direkt SPS-Signal	OPC UA der SPS	HTTP mit X-API-KEY
Latenz	< 10 ms	100-500 ms	500ms Loop + API
Fehleranalyse	Hardware vor Ort + TIA Portal	API-Statuscodes + Omniverse Konsole + TIA Portal	"TIMEOUT/HTTP" Labels + Omniverse Konsole
Fernsteuerung	Nur lokal	Nur im Netz der Anlage	Weltweit

Internal Block Diagram



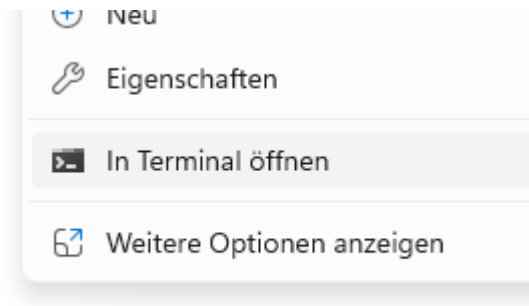
Sequenzdiagramm Variable Setzen

Digital Twin Sequence Diagramm



Isaac Sim Herunterladen

1. [Download Link](#) zu Isaac Sim
2. Entpacken der ZIP-Datei an einem gewünschten Installationsort (am besten Desktop)
3. „isaac-sim.bat“ per Doppelklick ausführen
Alternativ mit [Git-Bash](#)
4. Zu Zielordner Navigieren
5. Rechtsklick & „In Terminal öffnen“
6. `git clone https://github.com/NVIDIA-Omniverse/kat-app-template`
7. `cd kat-app-template`
8. `.\repo.bat template new`
9. `.\repo.bat build`
10. `.\repo.bat launch`



python_packages	Dateiordner
site	Dateiordner
standalone_examples	Dateiordner
tests	Dateiordner
tools	Dateiordner
clear_caches	Windows-Batchdatei
environment	Yaml-Quelldatei
isaac-sim	Windows-Batchdatei
isaac-sim.action_and_event_data_g...	Windows-Batchdatei
isaac-sim.compatibility_check	Windows-Batchdatei
isaac-sim.fabric	Windows-Batchdatei
isaac-sim.selector	Windows-Batchdatei
isaac-sim.streaming	Windows-Batchdatei
isaac-sim.xr.vr	Windows-Batchdatei



Abhängigkeiten installieren

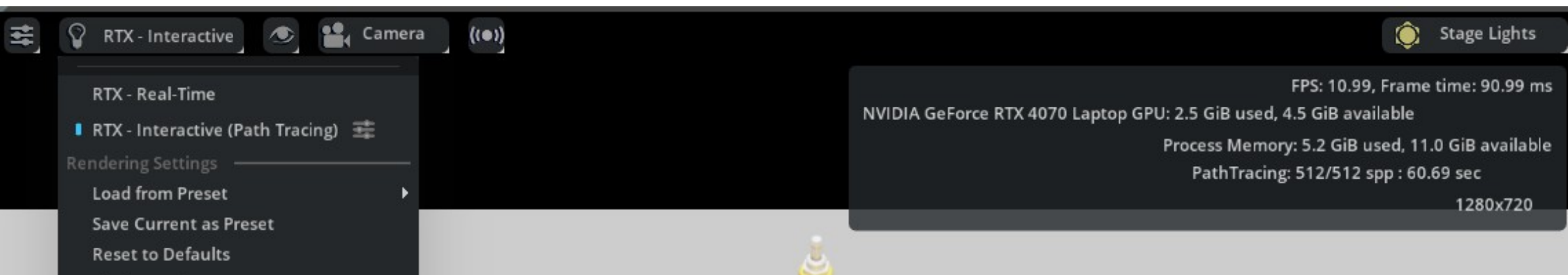
1. [NVIDIA Studio-Treiber 591.74](#)
2. [Python 3.11.15](#) (kann abweichen)
3. [CUDA 12.X](#) (eigentlich ist CUDA bei dem NVIDIA Treiber dabei, in der hier Dokumentierten Version musste CUDA 12.6 manuell installiert werden (Stand März 2026))
Die installierte Version kann im Terminal mit folgendem Befehl eingesehen werden:
`nvcc --version`
4. MQTT Library im Isaac SIM Installationsordner folgenden Befehl ausführen:
`.\python.bat -m pip install paho-mqtt==1.6.1`



Problembehandlung

1. Flackern der Szene (Z-Fighting)

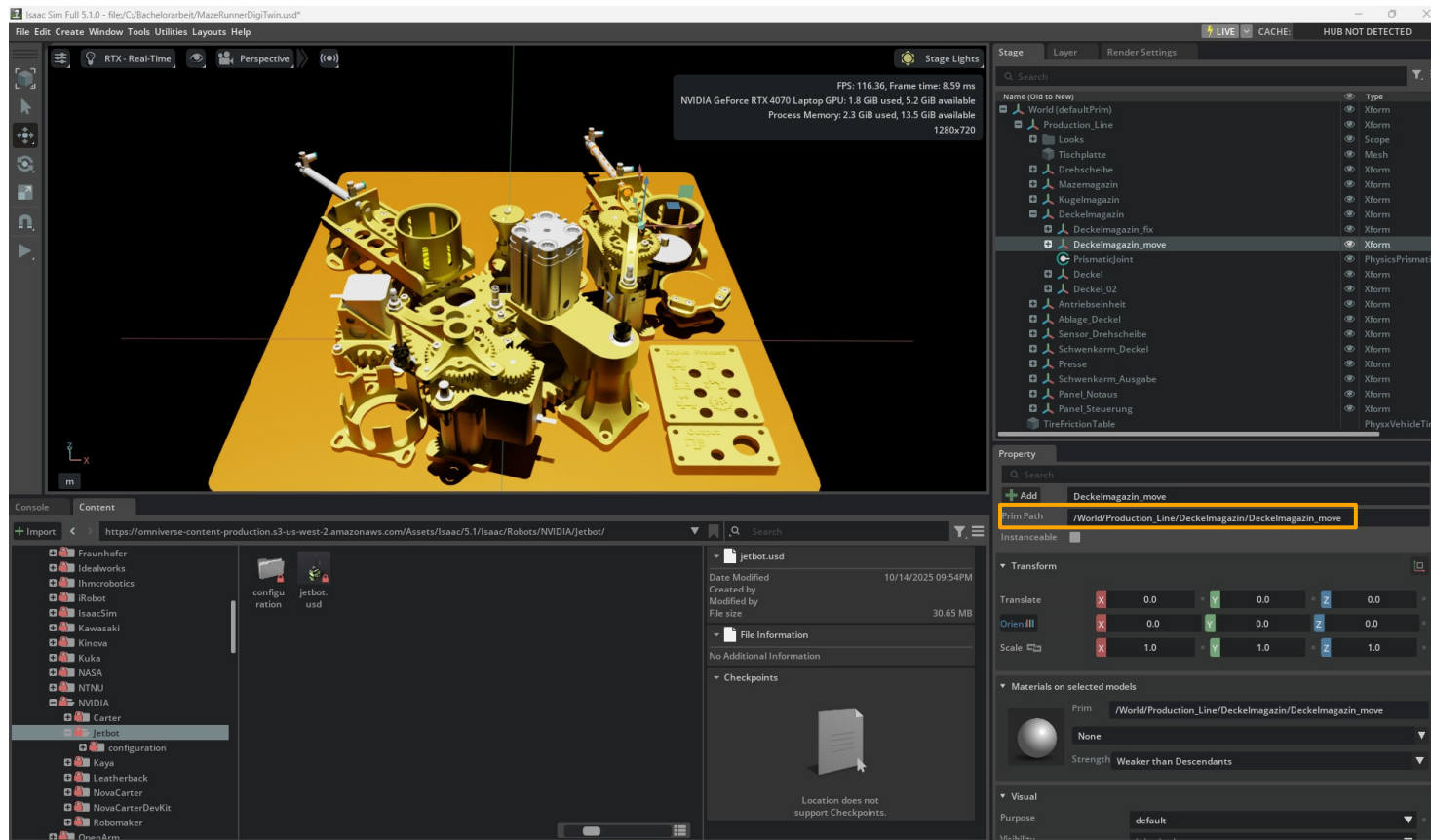
1. RTX – Interactive → RTX – Interactive (Path Tracing)
2. Sicherstellen, dass die Grafikkarte verwendet wird
3. Möglicherweise „kämpfen“ zwei Objekte die ineinander liegen darum, wer „oben“ liegt also sichtbar ist. → Stage → Strukturbaum prüfen



Erste Schritte in Isaac Sim

Stage: Modellansicht
(Rechtsklick → drehen)

Werkzeuge:
Verschieben
Skalieren
etc.
Play um die
Szene zu starten



Stage:
Ordnerstruktur der
Baugruppen-
komponenten
Sowie Beleuchtung
Material etc.

Property:
Eigenschaften des
ausgewählten
Modells
Prim Path: ist der
Pfad zu dem
ausgewählten
Objekt

Content: Dateiübersicht von Omniverse
Dateien oder Lokalen Projekten

STP-Datei in Isaac Sim öffnen

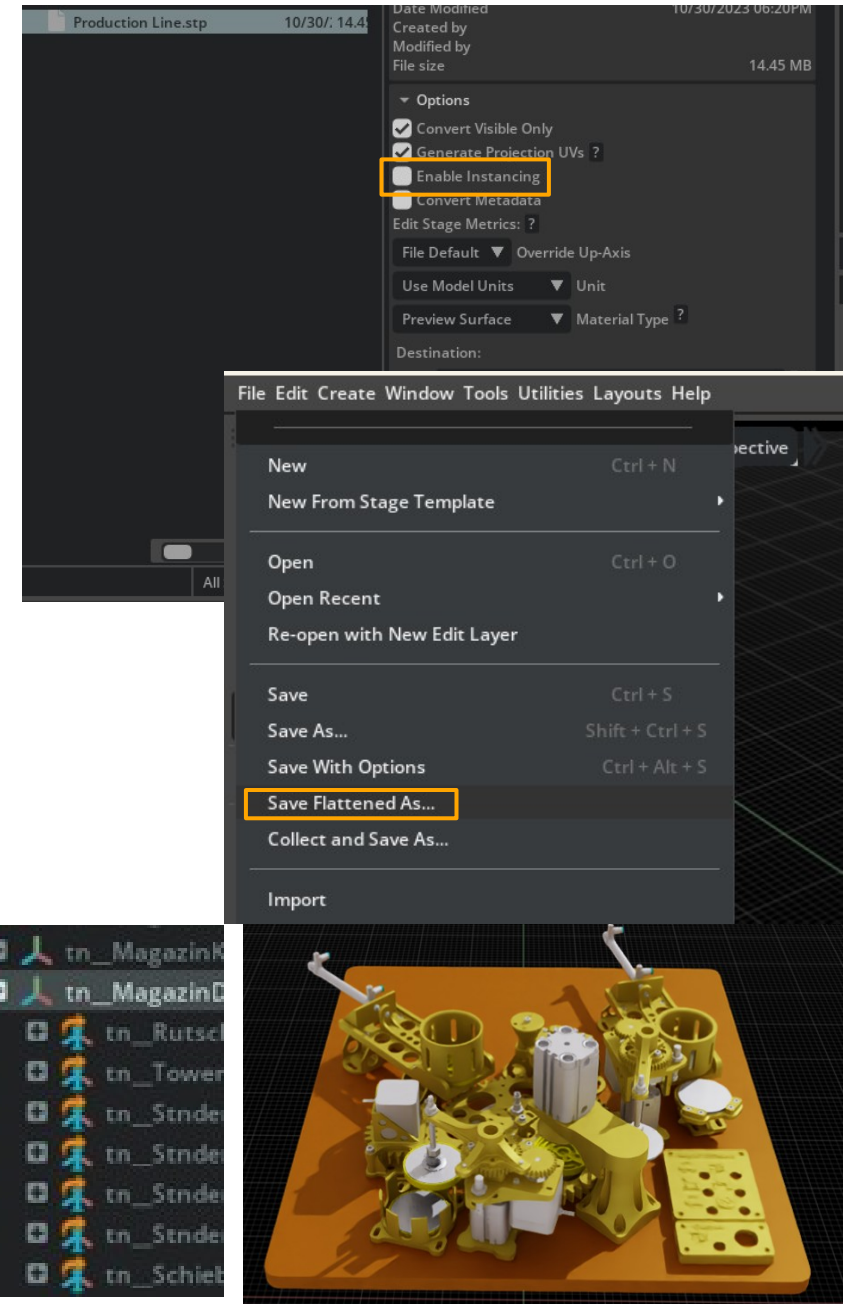
1. File → Import → .STP-Datei auswählen
Dabei „Enable Instancing“ deaktivieren
2. Um Abhängigkeiten zu anderen Modellen zu löschen:
File → „Save Flattened As...“
Sollte dieses ausgegraut sein, vorerst „Save as...“
3. Nach öffnen des gespeicherten Modells sollten die Farben übernommen worden sein
4. **Wichtig:** Im Stage Bereich dürfen weder blaue- noch orangefarbene Pfeile an den Einträgen sein.
Grund: Das würde bedeuten, dass dieses Teil mit einer Datei außerhalb des Projektes synchronisiert ist und nicht veränderbar ist.
Prüfen: Der Name sollte änderbar sein
Dieses Skript-Editor Skript entfernt alle orangefarbenen Haken →

```
import omni.usd
from pxr import Usd

# Hole die aktuelle Stage
stage = omni.usd.get_context().get_stage()

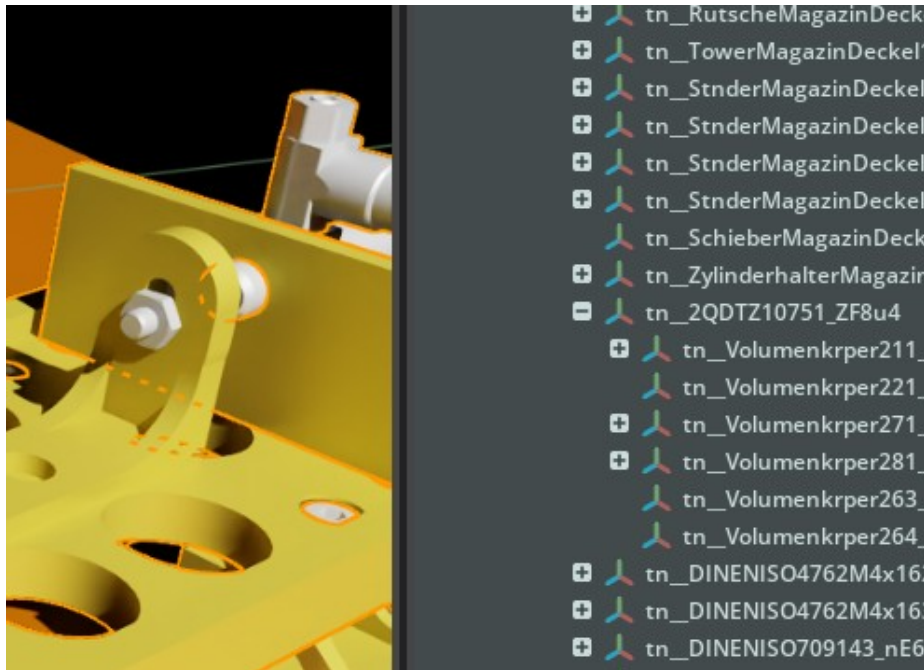
# Gehe durch alle Prims der Szene
for prim in stage.Traverse():
    if prim.IsInstancable():
        prim.SetInstancable(False)

print("Alle Instanz-Haken wurden entfernt.")
```



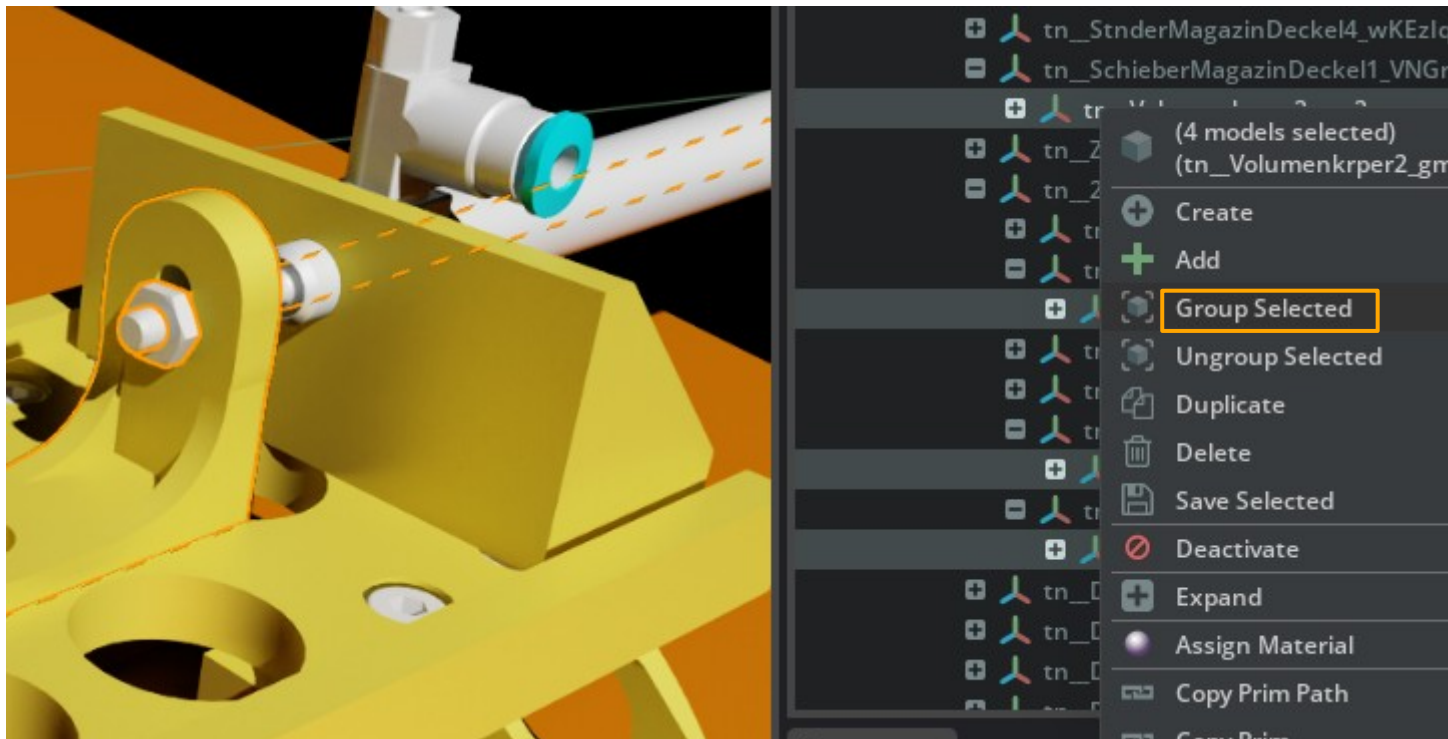
Starre Gelenkteile am Bsp. Magazin Glasplatte

1. Alle Teile auswählen die starr sein sollen
2. Rechtsklick auf eines der Teile und „Group Selected“ auswählen
3. Die erstellte Gruppe „Magazin_Glasplatte_fix“ benennen
4. Die finale Ordnerstruktur sollte wie folgt aussehen:



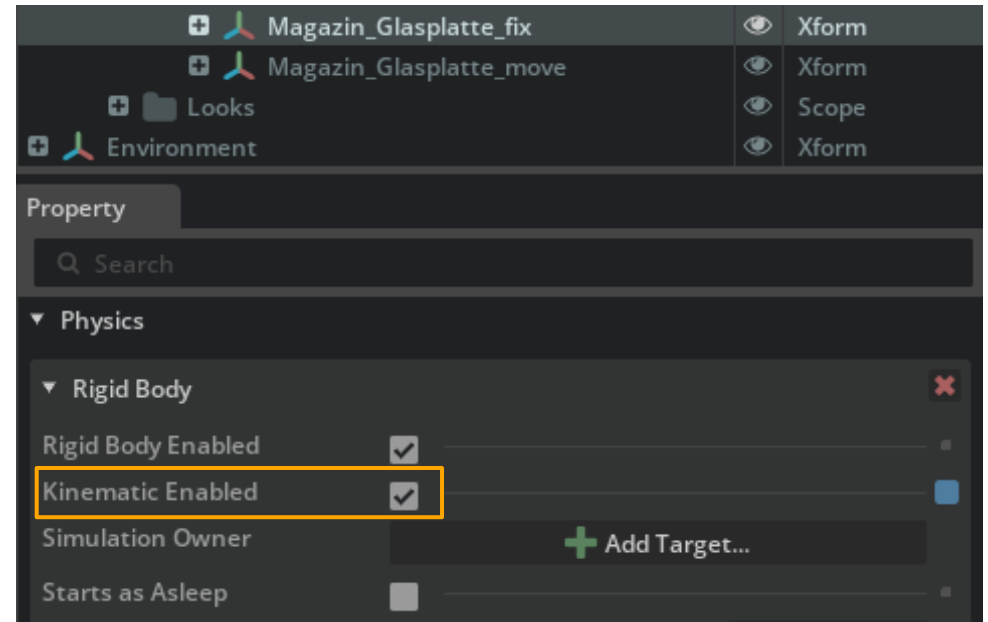
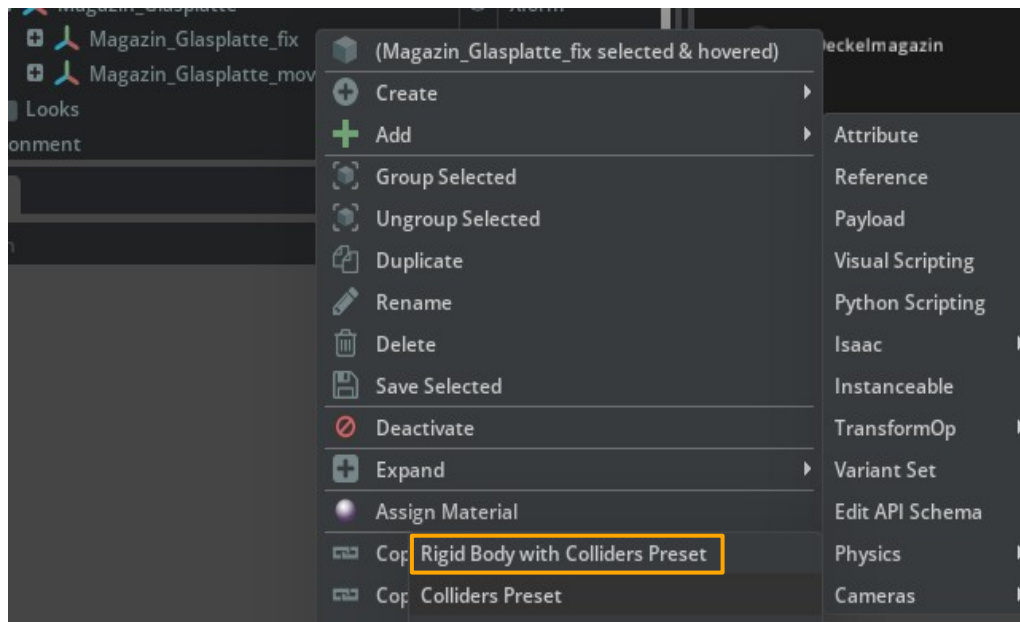
Bewegliche Gelenkteile am Bsp. Magazin Glasplatte

1. Strg-Taste halten und alle beweglichen Teile im Modell auswählen
2. Rechtsklick auf eines der Teile und „Group Selected“ auswählen
3. Die erstellte Gruppe „Magazin_Glasplatte_move“ benennen



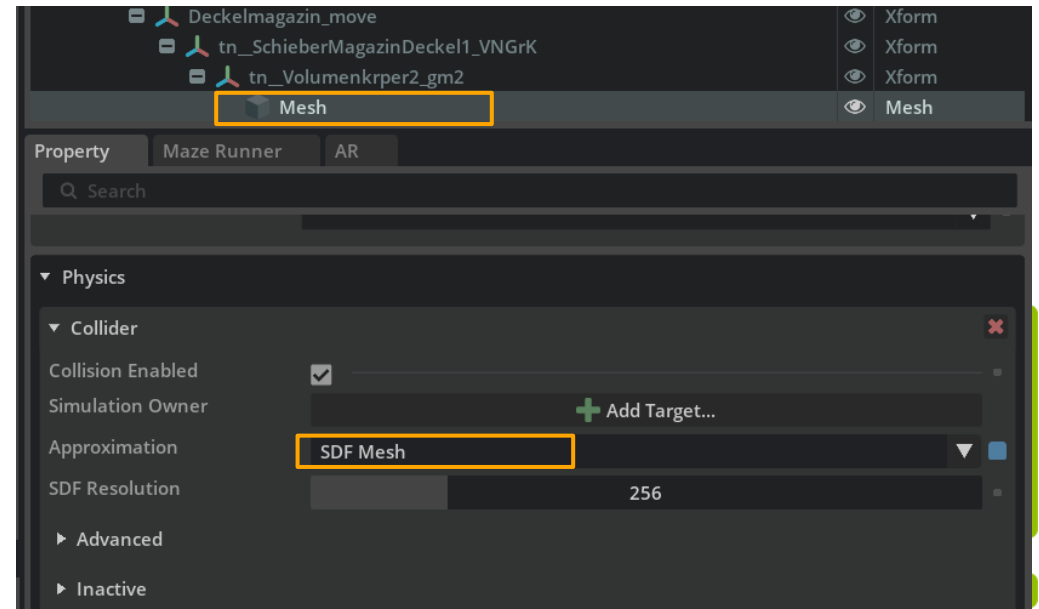
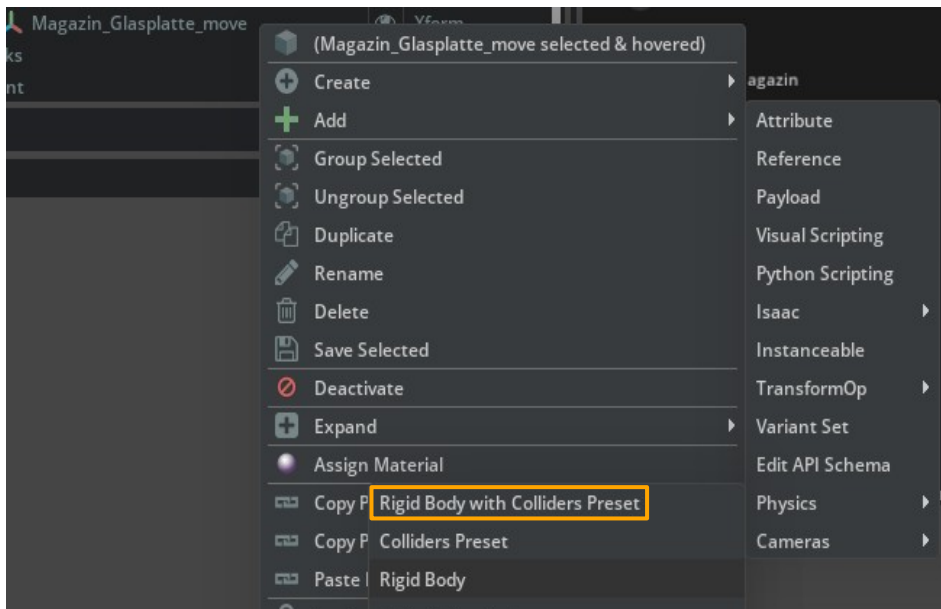
Physikalische Eigenschaften starrer Gelenkteile

1. Rechtsklick auf bewegliche Gruppe „Add“ → „Physics“ → „Rigid Body“
2. In Property unter „Rigid Body“ „Kinematic Enabled“ aktivieren
(So bleibt der Körper am Ort und fällt nicht durch die Grundplatte)



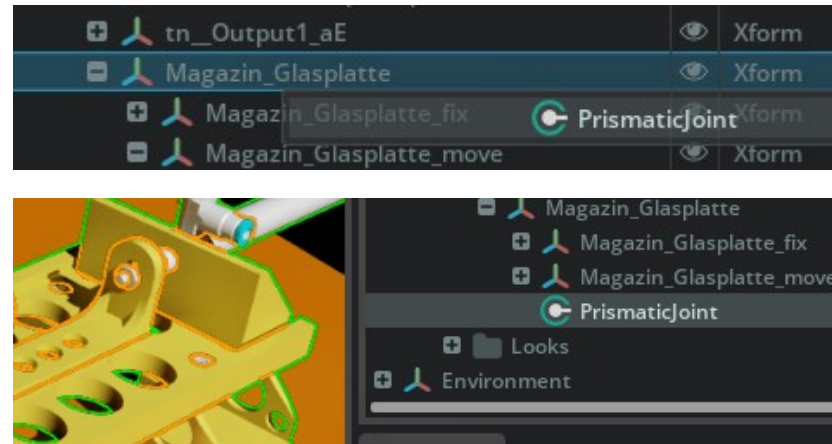
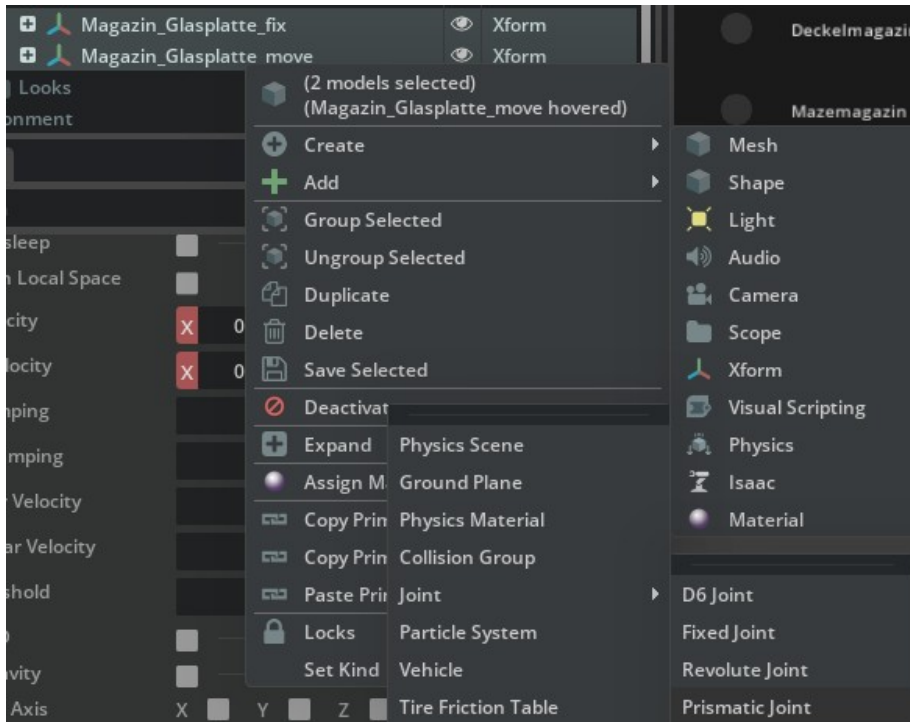
Physikalische Eigenschaften beweglicher Körper

1. Rechtsklick auf bewegliche Gruppe „Add“ → „Physics“ → „Rigid Body with Collider presets“
2. Der „Collider“ beschreibt die Hülle die um das Mesh gelegt wird und bei Berührung mit anderen Körpern zur Berechnung verwendet wird und kann unter „Physics“ der jeweiligen „Mesh Datei“ eingesehen werden.
3. Für die höchste Genauigkeit empfiehlt sich „SDF Mesh“ oder „Convex Decomposition“ im Feld „Approximation“ auszuwählen.
4. Dies gilt ebenfalls für frei bewegliche Körper wie der Deckel.



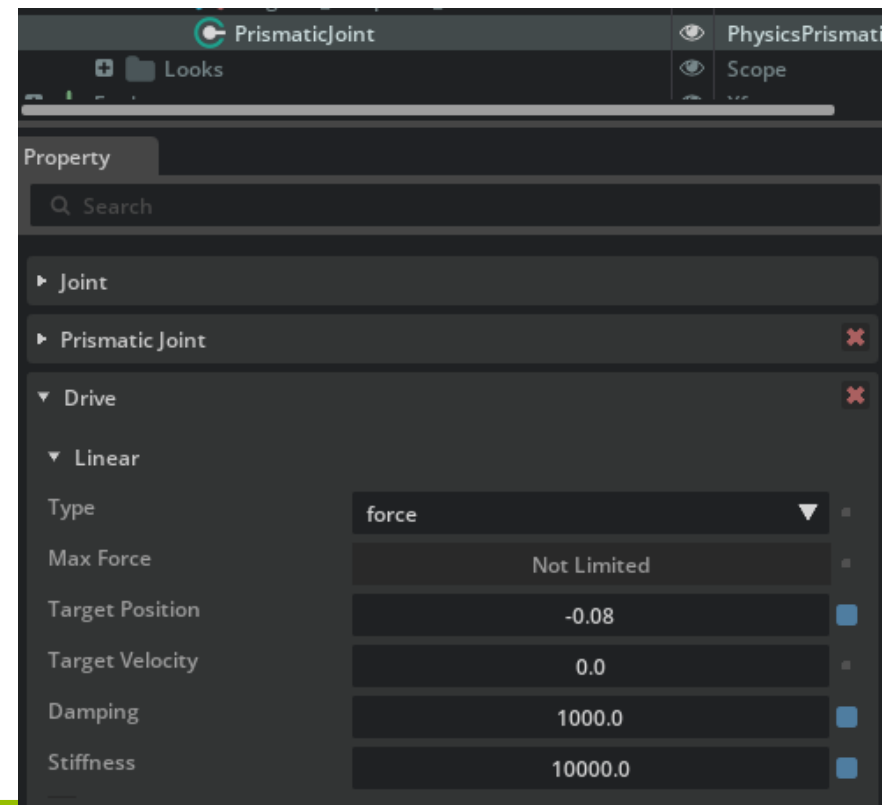
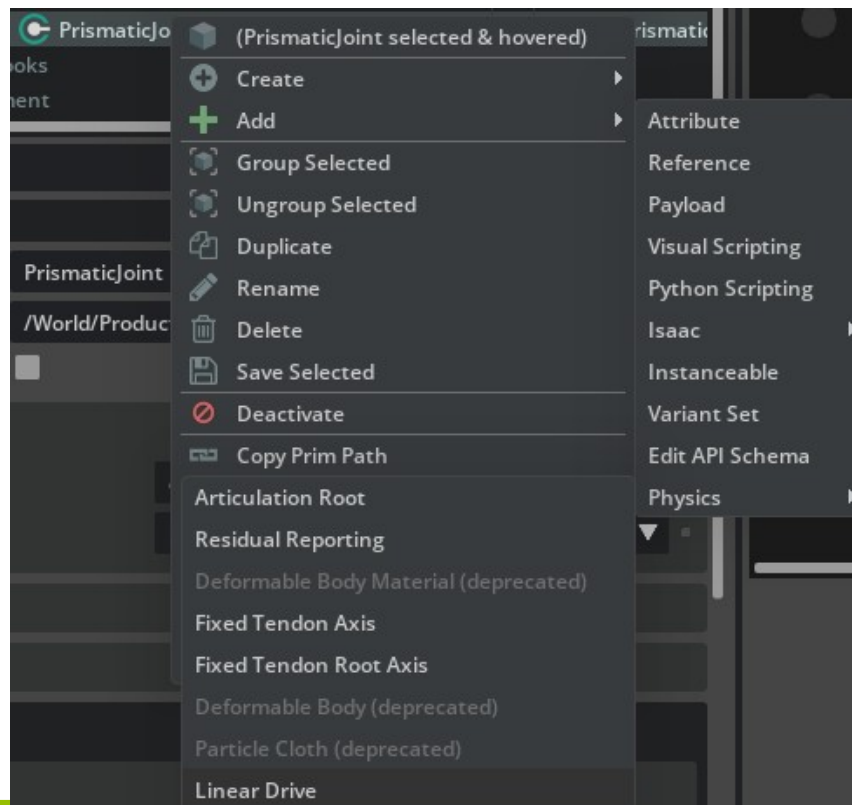
Gelenk (Joint) hinzufügen

1. Beide Unterordner auswählen (_move & _fix) → Rechtsklick → Create → Physics → Joint → Prismatic Joint
2. Joint Eintrag per Drag and Drop in Hauptordner ablegen
3. Bei Auswahl des Joints sollte das Modell wie hier farblich markiert werden



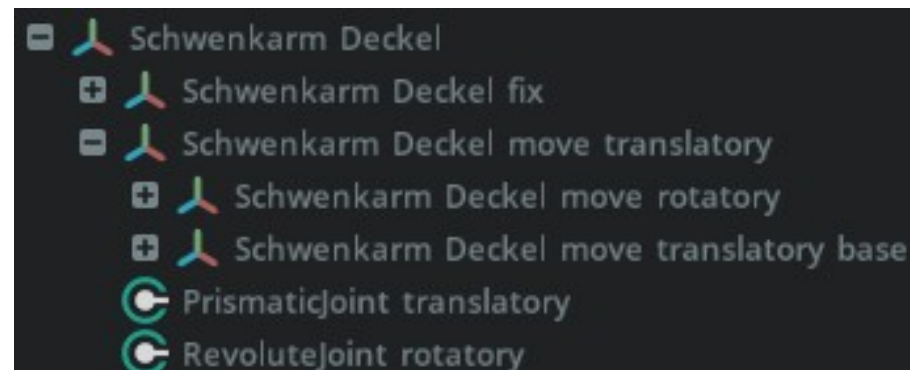
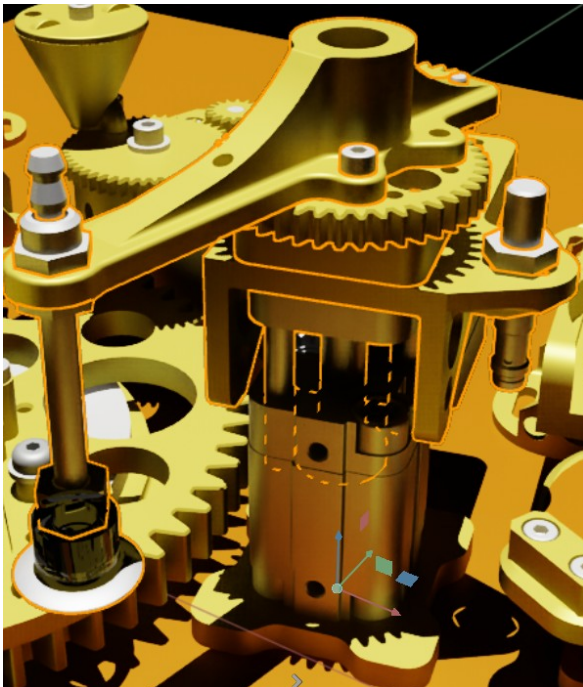
Antrieb einfügen

1. Rechtsklick auf Joint → „Add“ → „Physics“ → „Linear Drive“
2. In Property unter „Drive“ Werte wie im Bild übernehmen und ggf. anpassen
3. Funktion mit Play-Symbol oder Leertaste Testen



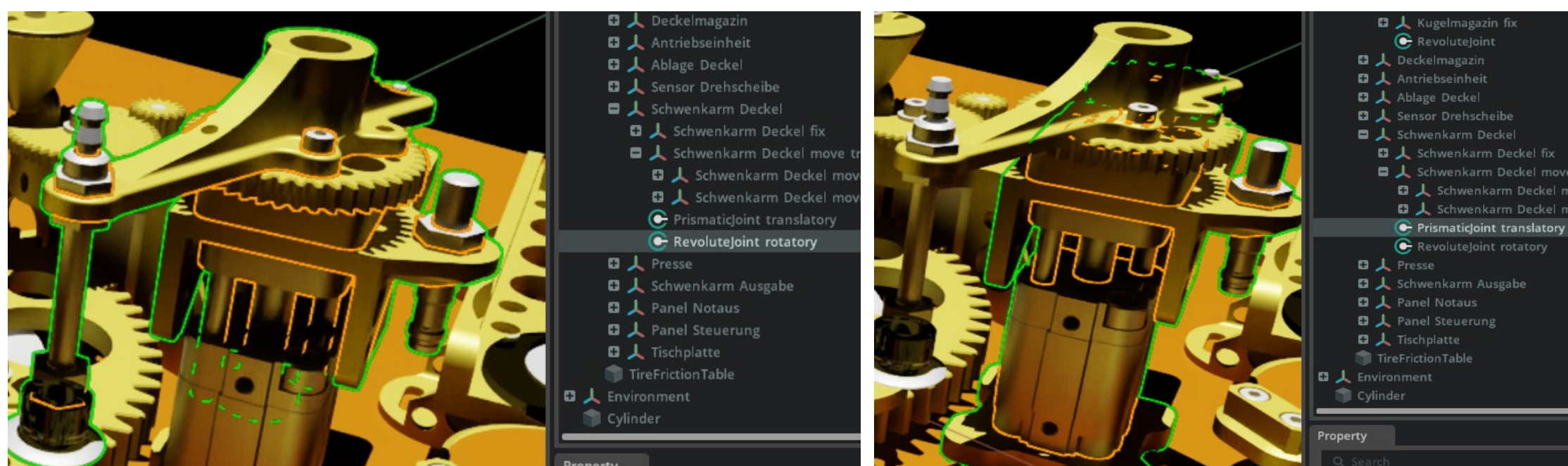
Verschachtelte Gelenke

1. Für Verschachtelte Gelenke gibt es Details die beachtet werden müssen
2. Diese werden im Folgenden am Beispiel des Sauggreifer Gelenkarms aufgezeigt
3. Dieser verfügt über ein Drehgelenk im oberen Teil und ein Schubgelenk für den oberen Aufbau mit der entsprechenden Ordnerstruktur



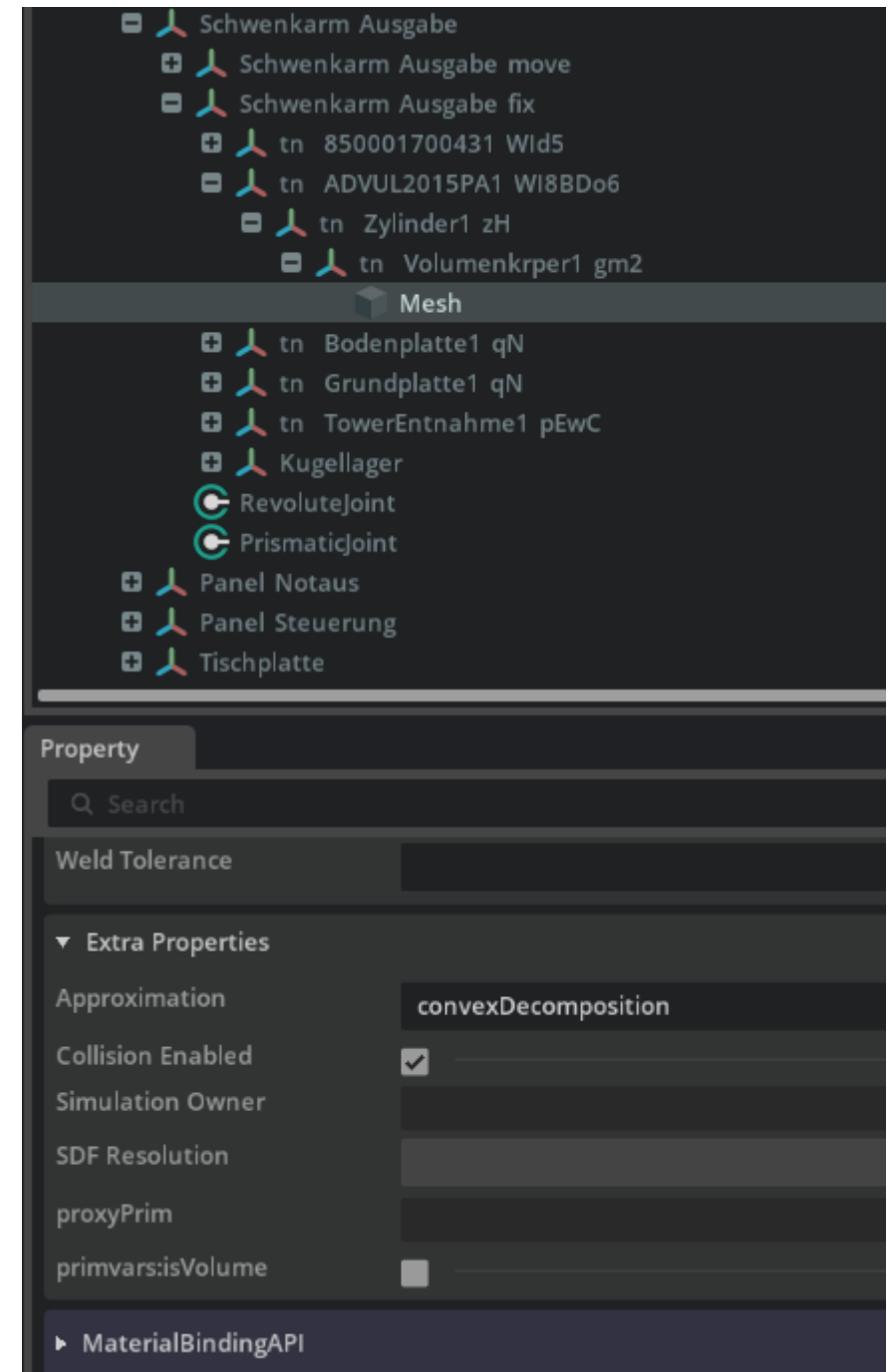
Verschachtelte Gelenke

1. Für die Drehbewegung wird ein Gelenk zwischen beiden Baugruppen mit der Bezeichnung „move“ erstellt.
2. Für die Schubbewegung darf ein Gelenk nur zwischen der „Base“ und der nicht drehenden Trägerbaugruppe erstellt werden.



Verschachtelte Gelenke

1. Das Mesh eines Körpers wird von der Physx Engine Approximiert.
2. „convexDecomposition“ löst das Mesh sehr genau auf und verhindert somit Kollisionen zwischen ungenauen Körpern welche über ihre Maße hinaus Approximiert wurden.



Extension Einleitung

Diese Extension ermöglicht den bidirektionalen Datenaustausch zwischen SPS und digitalem Zwilling in Isaac Sim zur Sicherstellung der Konformität der Aktorik. Über den direkten Zugriff auf Prim-Pfade der USD-Struktur werden komplexe Signale, wie inkrementelle Drehschritte oder getriggerte Baugruppen-Sequenzen, im Zwilling abgebildet. Eine Benutzeroberfläche bereitet die Prozessdaten auf und macht alle Funktionen manuell bedienbar.

Grundsätzlich kann die Extension sämtliche Operationen der Isaac Sim Oberfläche (z. B. Gelenkerstellung, Kollisionsprüfung) übernehmen, also jegliche Vorgänge die auch per Mausklick realisiert werden. Darüber hinaus können weitere, nicht Isaac Sim angehörige Komponenten hinzugefügt werden. Dazu zählen z.B. Verbindungen zu Webservern. Ergänzend ermöglicht ein Skript-Editor die direkte Code-Ausführung in der Szene für Einmal-Operationen, zu Testzwecken oder das Debugging. Ziel ist die Identifikation systemischer Problematiken beim Betrieb funktionaler Zwillinge, weit über die rein visuelle Darstellung hinaus. Im weiteren wird die Inbetriebnahme der Extension für die Maze Runner Anlage erklärt. Um diese auf eine andere Anlage übertragen zu können muss die Extension verstanden werden. Hier sind Large Language Modelle von großer Hilfe. Dabei sollte jedoch auf ein tiefgehendes Verständnis hingearbeitet werden, anstatt nur Anforderungen zu erfüllen.



constants.py

- Zentrale Konfigurationsdatei
- Farben für UI (Hintergrund, Akzent, Status)
- USD-Pfade (Deckel, Greifer, Joint, Maze)
- MQTT-Broker, Port, Keepalive
- API-URLs und API-Key
- Geometrie-Offsets (Deckelhöhe, Sicherheitsabstand)
- MAX_LOG_LINES = 80



logger.py

- `__init__(log_container, loop)` → Bindet UI-Container und Event-Loop
- `log(message, level)` → Fügt Zeile hinzu, thread-sicher
- `_rebuild()` → Zeichnet alle Log-Zeilen neu im UI
- `clear()` → Leert die Logliste
- Farben pro Level: log/info/error/ok
- Begrenzte Logtiefe (FIFO)



suction_gripper.py

- `attach()` / `detach()` / `toggle()` → Greiferzustand
- `hold_on_maze()` → Deckel über Maze fixieren
- `update_hold_position()` → Pro Frame nachführen
- `press_down(z_down)` → Deckel absenken (Press-Logik)
- `wait_and_press_if_ready(...)` → Auto-Press bei erreichter Position
- `release_dynamic()` → Deckel wieder physikalisch
- `reset()` → Joint entfernen, Zustand löschen
- Helpers: `_compute_maze_snap_world`, `_set_target_kinematic`, `_find_rigidbody_ancestor`



node_manager.py

- `load(on_button_clicked)` → JSON laden, UI-Zeilen erzeugen
- `find(node_id)` → Liefert Node-Definition
- `set_display(node_id, val)` → Label aktualisieren + USD setzen
- `apply_usd_for_node(node_id, val)` → USD-Attribut bei Toggle
- `set_usd_attr(stage, node, value)` → Schreibt USD-Attribut robust
- Datenfelder: `nodes`, `node_values`, `node_labels`
- Sonderzustände für Impuls- und Velocity-Modi



api_client.py

- `send_set(node_id, value, sim_mode)` → Setzt Bool via REST
- `send_impulse(node_id, sim_mode)` → True → 300ms → False
- `initial_poll(sim_mode)` → Alle Werte beim LIVE-Wechsel laden
- Im SIM-Modus nur lokale Updates
- Fehlerbehandlung mit Logging
- Nutzt aiohttp asynchron



mqtt_handler.py

- `start()` → Baut WebSocket-MQTT-Verbindung auf
- `stop()` → Disconnect + Statusupdate
- `_on_message(topic, payload)` → Thread-sicheres Routing in UI
- `_stepper_increment()` → Spezialfall Drehscheibe (-60° pro Trigger)
- Subscribed alle `/PlcNode/Get/<node_id>` Topics
- Bridge zwischen MQTT-Thread und asyncio-Loop



timeline_handler.py

- subscribe() → Bindet Omniverse-Timeline-Events
- unsubscribe() → Lösen + SIM-Loop stoppen
- _on_event() → Reagiert auf PLAY / STOP
- _sim_loop() → 20Hz Loop: Deckel halten + Presse prüfen
- _reset_all_to_zero() → Setzt alle Werte/Anzeigen zurück
- Hält Greifer-Verhalten konsistent mit Timeline



routines.py

- `start_gesamtprozess()` → Startet Hauptablauf (UI-Button)
- `_run_gesamtprozess()` → 11 Phasen Choreografie
- `_run_ba_start()` → Schwenkarm-Sequenz (Pickup → Place)
- `_step(description, delay)` → Loggt + wartet
- `_set(node_id, value)` → Setzt Node (Sonderfall Sauggreifer)
- `_impulse(node_id)` → Triggert STEP-Impuls
- Race-Schutz: `_gesamt_running`, `_ba_running`



ui_builder.py

- `build(callbacks...)` → Erzeugt Hauptfenster
- `_build_header()` → Titel + Statusanzeige
- `_build_toolbar()` → Refresh / Restart / SIM↔LIVE
- `_build_column_headers()` → Spaltenüberschriften
- `_build_gesamt_button()` → Großer Prozess-Start-Button
- `_build_log_header()` → Log-Bereich mit Clear-Button
- Liefert Referenzen für externen Logik-Controller



extension.py

- `on_startup(ext_id)` → Initialisiert alle Komponenten
- `on_shutdown()` → Sauberes Aufräumen, Tasks killen
- `_refresh_nodes()` → Lädt JSON neu
- `_restart_extension()` → Extension neu starten
- `_toggle_sim_mode()` → Wechsel SIM ↔ LIVE
- `_on_control_clicked(node_id)` → Routet Toggle/Step/Velocity
- `execute_step_impulse(node_id, node)` → Step-Impuls ausführen
- `_execute_velocity_impulse(node_id, node)` → Async Velocity-Impuls
- `_set_status_text(text, color)` → Statusbar oben rechts
- Zentraler Controller, der alle Module verbindet



Nodes_db.json

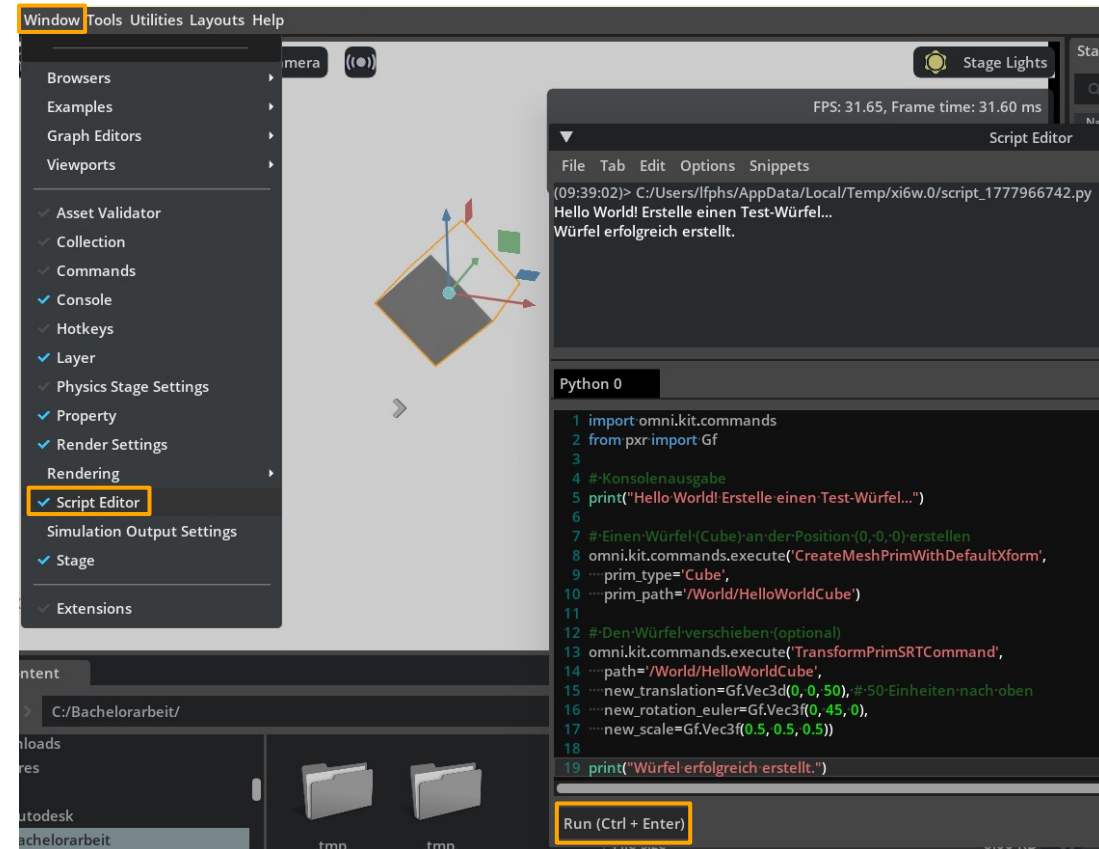
6. Neben der extension.toml liegen unter dem Pfad `omni\mazerunner\API` die .py - Dateien der Extension
7. In diesem Ordner befindet sich ebenfalls die Datei „Nodes_DB.json“
Hier werden die Nodes aus DigitalTwinApp mit der Omniverse Szene verknüpft dabei gilt folgendes:
 1. „display_name“ wird später im UI angezeigt
 2. „node_id“ muss mit DigitalTwinApp übereinstimmen
 3. „prim_path“ ist der Pfad zum Gelenk in Omniverse
 4. „attribute“ beschreibt den Zielparameter der manipuliert werden soll
 5. „target_value“ ist der Zielwert



```
{
  "nodes": [
    {
      "display_name": „BM_MoveFront_Set“,
      "node_id": " BM_MoveFront_Set ",
      "prim_path": "/World/Robot/Base_Joint",
      "attribute": "drive:angular:physics:targetPosition",
      "target_value": 90.0
    }
  ]
}
```

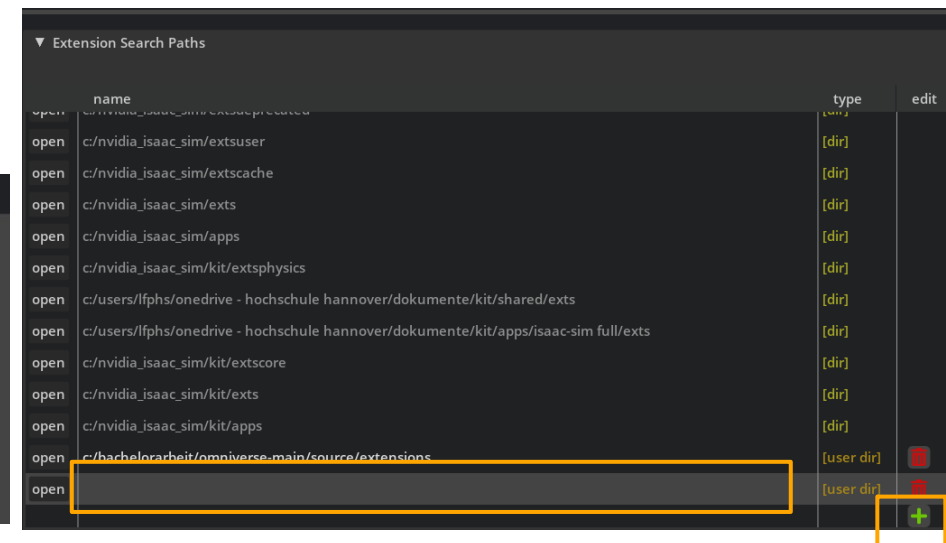
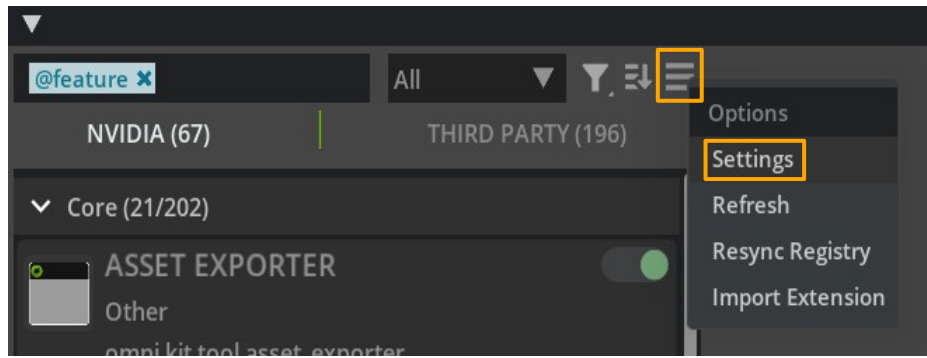
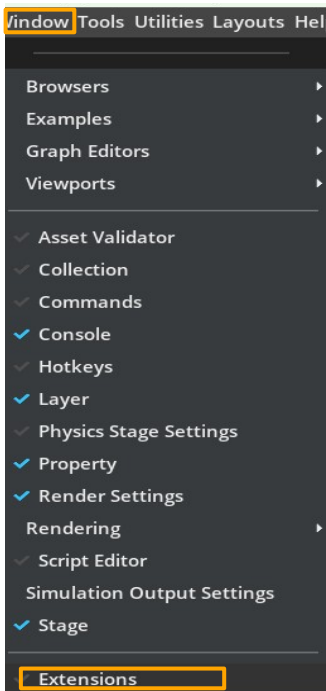
Skript Editor

1. Der Skript Editor befindet sich unter Window → Skript Editor
2. Der Code wird in das Schwarze Fenster geschrieben und mit Enter ausgeführt.
3. Zur Demonstration wurde hier ein Würfel mit dem Namen „Hello World“ durch den Skript Editor erstellt



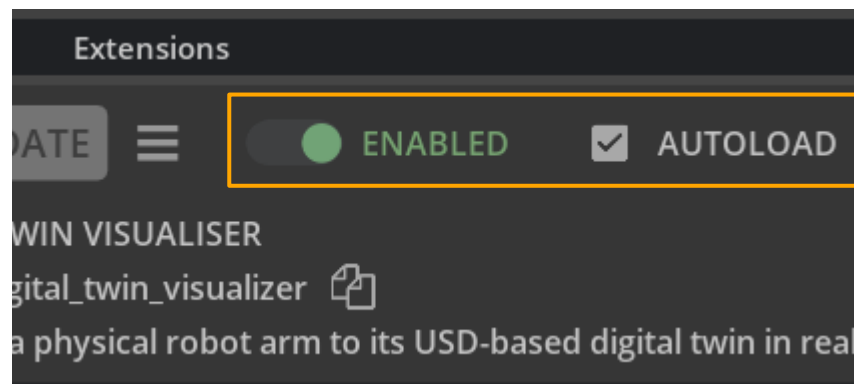
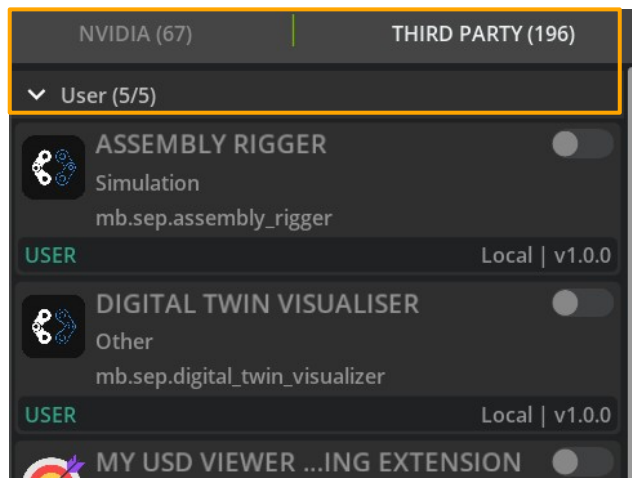
Extension aktivieren

1. Extension Datei herunterladen und in eine Ordner entpacken.
2. In Isaac Sim unter Window → Extensions auswählen.
3. Im neuen Fenster die drei Striche → Settings auswählen.
4. Hier auf das grüne Plus klicken und den Pfad zum Ordner angeben in dem die Extension liegt.



Extension aktivieren

1. Jetzt sollte im Bereich „Third Party“ unter User die neue Extension erscheinen.
2. Diese wird ausgewählt und im rechten Bereich auf „enabled“ und „autoload“ gestellt
3. Sollte es hier Probleme geben, gilt es die extension.TOML zu überprüfen.



Extension bearbeiten

1. [VS-Code](#) herunterladen
2. In VS Code durch die Datei, bis zu den Python Dateien, navigieren und diesen Ordner öffnen.
3. Die Textdatei extension.toml dient als „Visitenkarte“ der Extension für die Anzeige in Isaac Sim folgender Inhalt:

Wenn die Extension beim Aktivieren Fehlerhaft ist, liegt das meistens an dieser Datei.

```
[package]
title = "Maze Runner UI"
version = "1.0.0"

[dependencies]
"omni.kit.uiapp" = {}
"omni.ui" = {}

[[python.module]]
name = "omni.mazerunner.API"
path = "."
```



Extension Quellcode Imports

Bibliothek	Beschreibung
omni.ext	Basis-Klasse (IExt) für die Erstellung und Registrierung der Omniverse Extension.
omni.ui	Framework für das User Interface (Fenster, Buttons, Labels, ScrollingFrames).
omni.usd	Schnittstelle zur 3D-Szene, um auf USD-Prims und deren Attribute zuzugreifen.
os	Handhabung von Dateipfaden (z. B. Lokalisierung der nodes_db.json).
json	Parsen der Konfigurationsdatei und Verarbeiten der API-Antworten.
asyncio	Verwaltung asynchroner Tasks, um die Simulation nicht zu blockieren.
aiohttp	Asynchroner HTTP-Client für die Kommunikation mit der Web-API.

Funktionale Emulation mittels OPC UA Mock-Server

- Zweck: Bereitstellung einer virtuellen Gegenstelle (Server), die das Kommunikationsverhalten einer echten SPS simuliert.
- Vorteil: Entwicklung der Isaac Sim Extension unabhängig von physischer Hardware und Netzwerk-Infrastruktur.
- Test-Logik: Der Mock-Server liest die nodes_db.json aus und schaltet die hinterlegten Datenknoten (Nodes) zyklisch zwischen den Zielwerten (z. B. 0 und 1) um.
- Ziel: Validierung, ob die Extension die Werte korrekt empfängt und die digitalen Zwillinge in der Simulation entsprechend ansteuert.

```
MOCK SERVER: INITIALISIERUNG
-----
[BOOL] Registriert: BA_Start          | ID: ns=4;i=4
[BOOL] Registriert: BM_MoveFront_Set | ID: ns=4;i=19
[DOUBLE/FLOAT] Registriert: DS_StartStepper_Set | ID: ns=4;i=7
[BOOL] Registriert: DM_MoveFront_Set | ID: ns=4;i=10
[BOOL] Registriert: KM_Stepper_Start | ID: ns=4;i=13
[BOOL] Registriert: Squeezer_start_set | ID: ns=4;i=16
-----
SIMULATION GESTARTET (Schrittkette läuft...)
-----
>>> SET: BA_Start -> TRUE
<<< RESET: BA_Start -> FALSE
>>> SET: BM_MoveFront_Set -> TRUE
<<< RESET: BM_MoveFront_Set -> FALSE
>>> SET: DS_StartStepper_Set -> 90.0°
```

Property Maze Runner - OPC UA Live Monitor			
OPC UA Live-Monitor			REFRESH
Variable	IP	Node-ID	Zustand (Wert)
BA Start	192.168.60.12	ns=4;i=4	-
BM MoveFront Set	192.168.60.11	ns=4;i=19	FALSE
DS StartStepper Set	192.168.60.11	ns=4;i=7	0.00
DM MoveFront Set	192.168.60.11	ns=4;i=10	FALSE
KM Stepper Start	192.168.60.11	ns=4;i=13	TRUE
Squeezer start set	192.168.60.12	ns=4;i=16	FALSE
AA Start	192.168.60.12	ns=4;i=4	FALSE



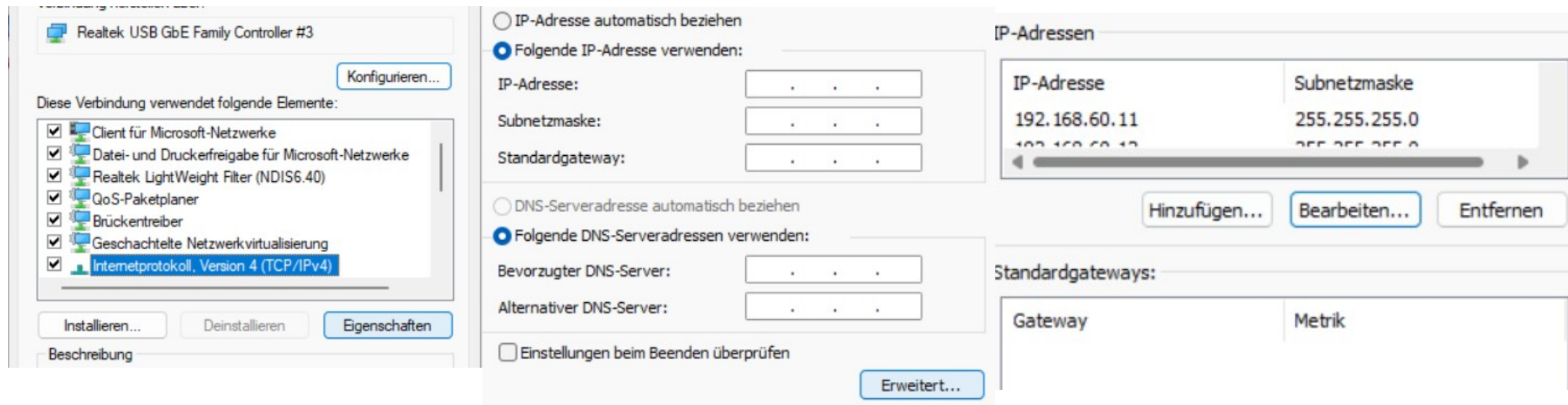
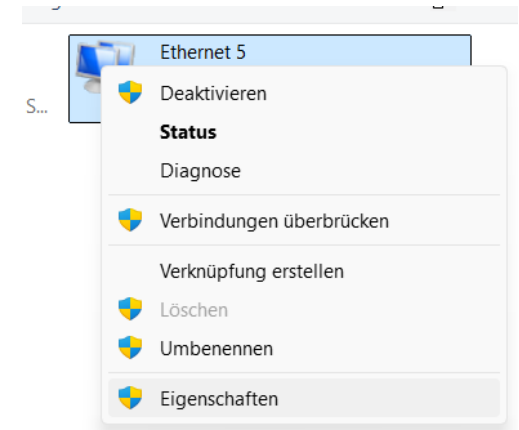
Implementierung des OPC UA Mock-Server

- IP-Adress-Zuweisung: Dem PC werden manuell die IP-Adressen der zukünftigen Hardware-Steuerungen zugewiesen (z. B. 192.168.60.11 und .12).
- Lokales Routing: Durch diese Zusatz-IPs "glaubt" das Betriebssystem, selbst die SPS-Gegenstelle zu sein.
- Kommunikationsfluss: Isaac Sim sendet Anfragen an die IP .11 oder .12. Der lokal laufende Mock-Server fängt diese Anfragen ab und antwortet mit den Test-Werten aus der Datenbank.
- Ergebnis: Die gesamte Kommunikationskette wird unter realen Netzwerkbedingungen (IP-basiert) getestet, ohne den Code später ändern zu müssen.



Leitfaden zu Inbetriebnahme des OPC UA Mock-Servers

1. Dazu Win-Taste → „Netzwerkverbindungen Anzeigen“
→ Rechtsklick auf Aktive Verbindung → „Eigenschaften“
2. „Internetprotokoll, Version 4“ → „Eigenschaften“
3. „Erweitert“ auswählen und die IP-Adressen der SPS-Anlagen einfügen
4. Eine Weitere IP-Adresse für den PC hinzufügen 192.168.60.13
5. Standardgateway freilassen



Transition zur realen Hardware

- IP-Adress-Bereinigung: Die zusätzlichen IP-Adressen (.11, .12) werden am PC in den Netzwerkeinstellungen gelöscht, um IP-Konflikte mit der echten Hardware zu vermeiden.
- Netzwerk-Kopplung: Physikalische Verbindung des PCs mit dem Automatisierungs-Netzwerk (via GL.iNet Router via Wifi).
Der PC behält dabei eine eigene, eindeutige IP (z. B. .13).
- Hardware-Aktivierung: Start des OPC UA Servers auf der realen SPS.
- Funktionstest: Da die `node_id` und die IP-Adressen in der `nodes_db.json` bereits identisch mit der Hardware konfiguriert wurden, ist keine Änderung am Programmcode der Extension erforderlich.



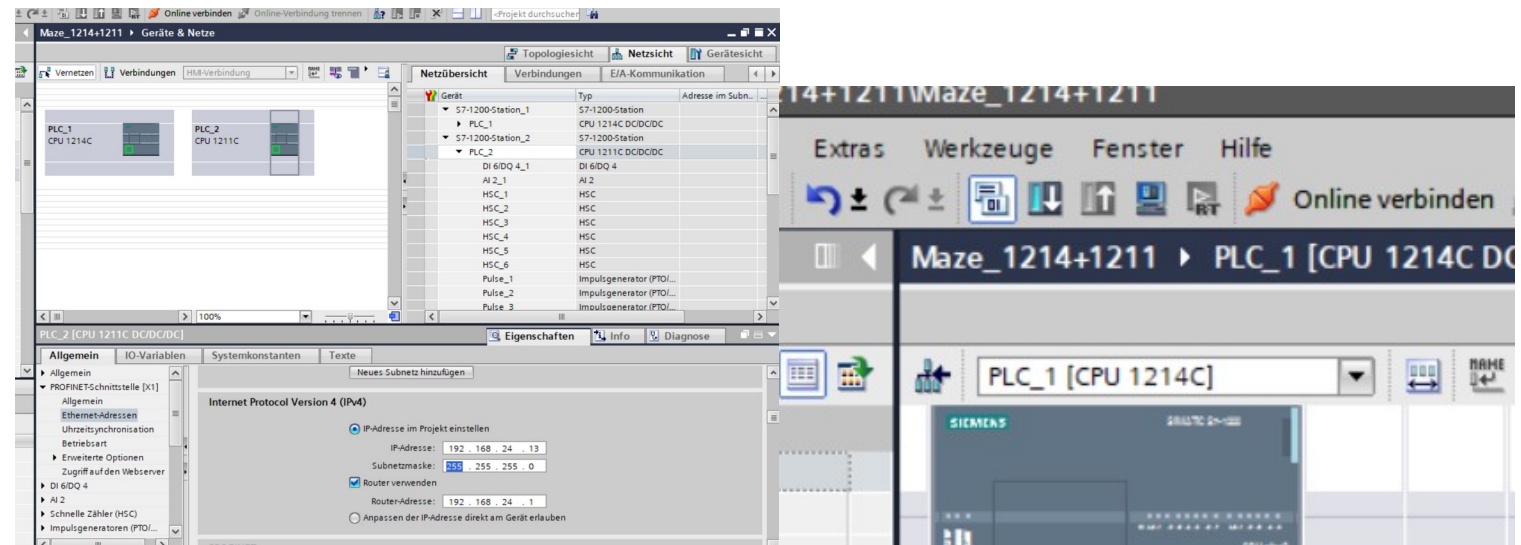
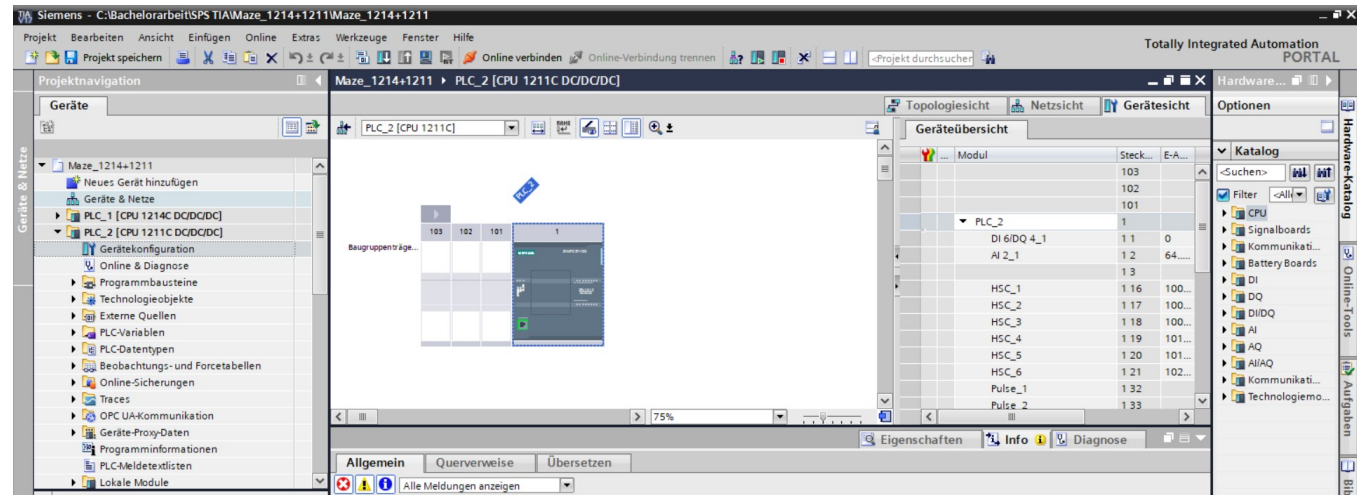
Installation

1. [TIA Portal Update 9](#) herunterladen
2. Falls [SIMATIC S7 Distributed Safety](#) beim Öffnen von TIA Portal als fehlend angezeigt wird, muss der Zugriff für den Download bei Siemens angefragt werden.

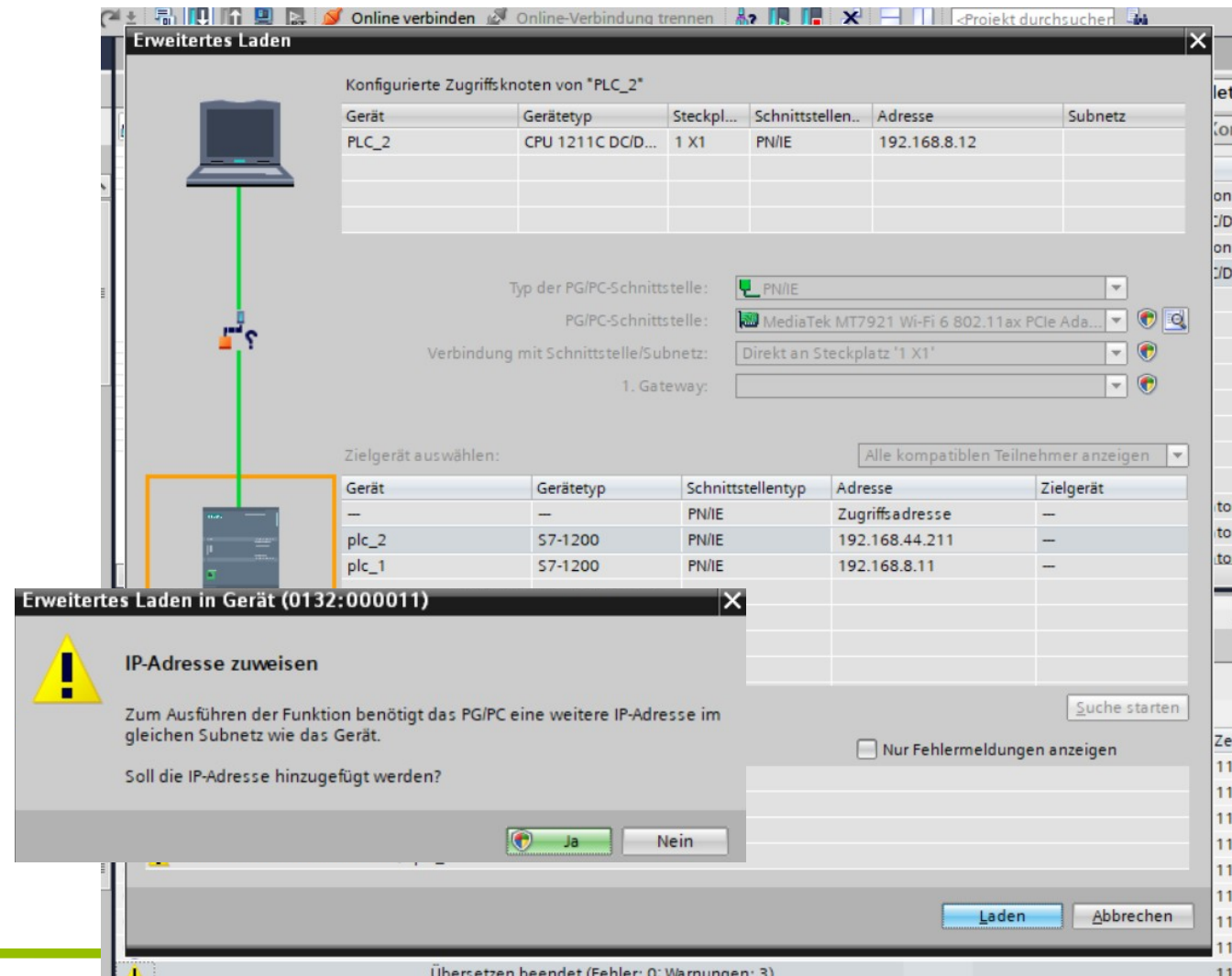


IP-Adresse der SPS Ändern

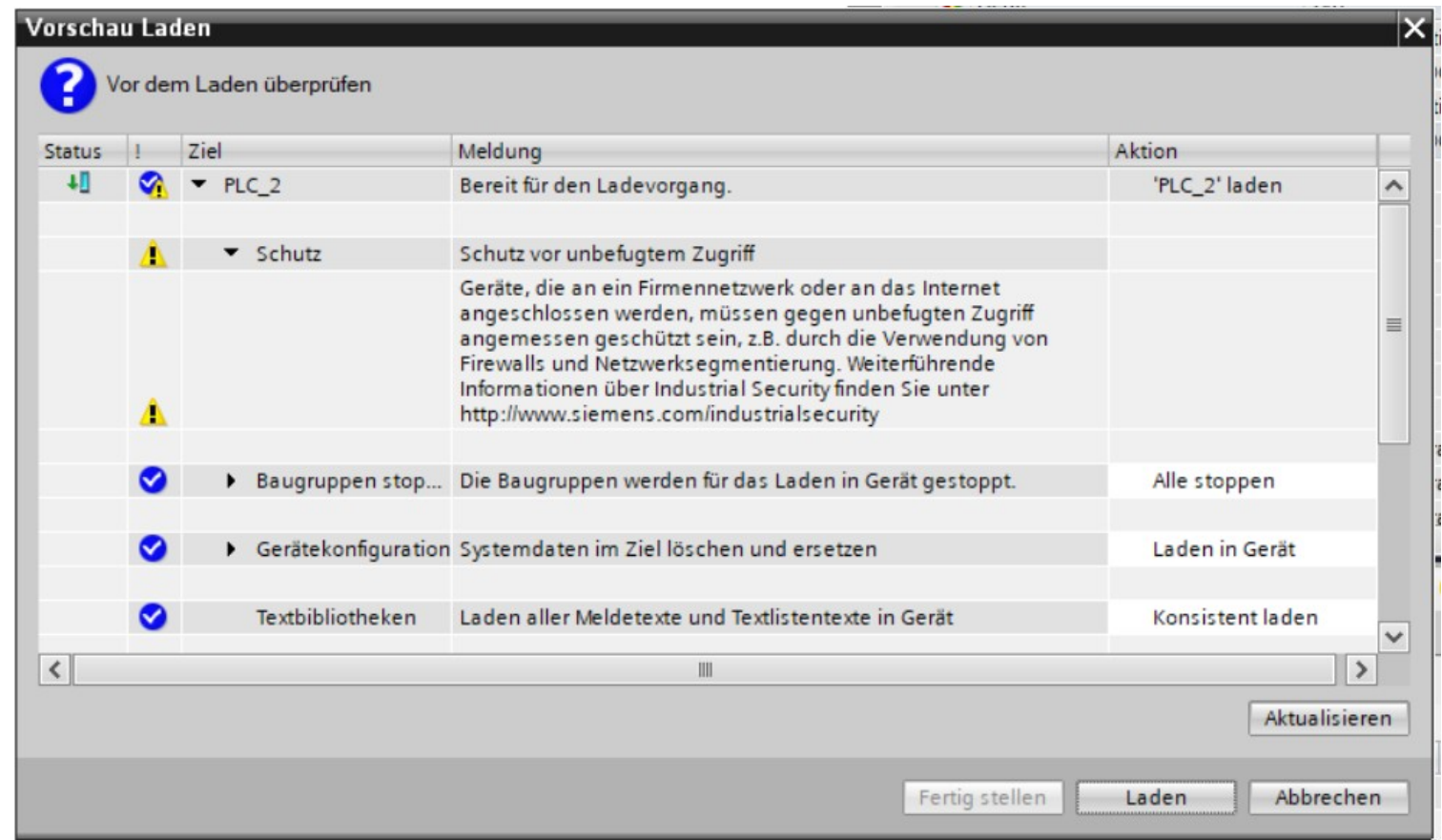
1. Online Verbindung Trennen



Verbindung zur SPS herstellen



Konfiguration Laden



MQTTL für SPS nachrüsten

[https://support.industry.siemens.com/cs/document/109780503/simatic-s7-1500-s7-1200-bibliotheken-für-kommunikation-\(lcommsuite\)?dti=0&lc=de-DE](https://support.industry.siemens.com/cs/document/109780503/simatic-s7-1500-s7-1200-bibliotheken-für-kommunikation-(lcommsuite)?dti=0&lc=de-DE)



Problembehandlung

1. Falls TIA
2. Falls [SIMATIC S7 Distributed Safety](#) beim Öffnen von TIA Portal als fehlend angezeigt wird muss der Zugriff angefragt werden.



Zusammenfassung

Die Maze-Runner-Extension wurde in elf klar abgegrenzte Module zerlegt, sodass jede Datei genau eine Verantwortung trägt und gut wartbar ist. Die `constants.py` bündelt sämtliche Konfigurationswerte sowie Farben, USD-Pfade für den Sauggreifer, Endpunkte und Geometrie-Offsets an einer zentralen Stelle. Die UI wird vollständig vom `UI_builder` aufgebaut, welcher die Layout-Logik liefert und Callbacks über Parameter entgegennimmt, dadurch bleibt das Hauptmodul frei von UI-Code. Die Datenhaltung übernimmt der `NodeManager`, der die JSON-Konfiguration lädt, Statuslabels pflegt und auf USD-Attribute schreibt. Spezialisierte Module kümmern sich um den Sauggreifer (`SuctionGripper`), die REST-API (`APIClient`), die Live-Verbindung über WebSocket-MQTT (`MQTTHandler`) und die Reaktion auf Omniverse-Timeline-Events (`TimelineHandler`). Der Gesamtprozess und die `BA_Start`-Sequenz liegen gekapselt in `routines.py`, was sie unabhängig vom Hauptmodul testbar und erweiterbar macht. Der zentrale Controller ist `extension.py`, er instanziiert die Komponenten, verbindet die UI-Callbacks und vermittelt Klicks an die jeweils zuständigen Module. Das Logging läuft thread-sicher über die `Logger-Klasse`, die sowohl aus dem `asyncio-Loop` als auch aus `MQTT-Threads` sicher angesprochen werden kann. Dadurch entsteht eine klare Trennung zwischen Darstellung, Datenmodell, Simulationslogik, externer Kommunikation und Automatisierung. Ein gut wartbares, erweiterbares Gesamtkonzept wobei noch keine zuverlässige MQTT Kommunikation dokumentiert werden kann, hier steht noch aus ob die SPS entsprechend konfiguriert werden muss, allerdings kommen vereinzelte, erwartete Nachrichten, stark verzögert in Isaac Sim an.



Quellen

1. Labor Mechatronik (Trainingslevel Konstruktion und Inbetriebnahme einer Maze Runner Produktionslinie unter Einsatz interoperabler Edge- und Cloud-Technologien) SS2024 (13.05.2026)
2. [Isaac Sim API Spezifikation](#) (12.04.2026)
3. [Isaac Sim Core](#) (01.05.2026)
4. [Isaac Sim Python API](#) (06.05.2026)
5. [Eclipse Paho MQTT Python Client](#) (13.05.2026)
6. [API – REST Requests](#) (13.05.2026)
7. [WebSocket API](#) (09.04.2026)
8. [OPC-UA Server Konfiguration](#) (18.02.2026)

[Download aller hier dokumentierter Inhalte](#)

